

FORMATION EMBARQUÉE

Guide de la formation

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Formation complète — Systèmes embarqués & programmation bas niveau

 **Nouveau ici ?** Commence par le **Parcours type** (PDF : [pdf/00-COMMENCER-ICI.pdf](#)) : les 30 premières minutes, la boucle de travail, le plan semaine par semaine et la checklist à imprimer.

Une formation structurée comme un enseignement universitaire/professionnel : **cours magistraux** (les modules), **travaux dirigés corrigés** ([td/](#)), **travaux pratiques guidés pas à pas** ([tp/](#)) et **évaluations** avec barème ([evaluations/](#)). Du transistor jusqu'aux automates industriels Siemens et Schneider.

Organisation pédagogique

Élément	Dossier	Rôle
Cours (11 modules)	cours/	La théorie, les exemples commentés, les concepts
Mini-TP sur simulateur	mini-tp/	Programmes à trous de 15-30 min, testables sans matériel sur Wokwi, OnlineGDB, EDA Playground, PLCSIM... — avec tous les liens des simulateurs
TD — travaux dirigés	td/	Exercices d'application avec corrigés détaillés et commentés
TP — travaux pratiques	tp/	Séances guidées pas à pas sur projet réel (matériel ou simulateur) ; chaque TP existe en fiches minutées séance par séance (tp/tpN-fiches/)
Code source des corrigés	code/	Les corrigés en vrais projets compilables et testés (make test) : C, C++, Java, VHDL (GHDL), Arduino, Python/Modbus, SCL/ST
Évaluations	evaluations/	QCM corrigé, une épreuve pratique par plateforme (eval-*.md — on n'évalue pas du Siemens comme de l'Arduino !) et projets finaux avec barème
Antisèches	antiseches/	7 mémos à imprimer : C, C++, VHDL, Arduino/STM32, automates, Git/terminal, conversions
Glossaire	glossaire.md	Tous les sigles du domaine (+ les « faux amis » qui piègent)
FAQ & dépannage	faq-depannage.md	Les blocages réels, outil par outil, avec leur solution
Suivi de progression	suivi-progression.md	Tableau de bord à imprimer : modules, TP, évaluations, habitudes, portfolio
Guide du formateur	guide-formateur.md	Pour animer la formation : planning type, matériel par poste, modalités d'évaluation, conseils d'animation, découpage en sessions courtes

Commandes utiles (depuis ce dossier) :

```
make verif    # vérifie tous les liens des documents
make test     # compile et exécute tous les corrigés (C, C++, Java, VHDL)
make pdf      # régénère les PDF           make zip  archive des PDF
```

Volume horaire indicatif (rythme ~6 h/semaine) :

Module	Cours	TD	TP	Total
00 Fondamentaux	6 h	4 h	—	10 h
01 Langage C	12 h	10 h	—	22 h
02 C++	8 h	6 h	—	14 h
03 Arduino	6 h	4 h	12 h (TP1)	22 h
04 VHDL	10 h	8 h	8 h (TP2)	26 h
05 Java	8 h	6 h	—	14 h
06 Autres langages	6 h	3 h	—	9 h
07 Siemens	10 h	6 h	10 h (TP3)	26 h
08 Schneider	8 h	6 h	8 h (TP4)	22 h
10 STM32	10 h	6 h	10 h (TP5)	26 h
Total	≈ 190 h (~8 mois à 6 h/sem.)			

Méthode de travail conseillée pour chaque module : 1. Lire le cours en entier une première fois (sans coder). 2. Relire en tapant et exécutant chaque exemple. 3. Faire le **mini-TP du thème** sur simulateur ([mini-tp/](#)) : 15-30 min pour valider les concepts clés avant d'aller plus loin. 4. Faire les exercices du cours **sans regarder** le TD. 5. Confronter au corrigé du TD ([td/td-NN-*.md](#)) — comprendre chaque écart. 6. Faire le TP associé s'il existe. 7. Valider avec le QCM du module ([evaluations/qcm.md](#)) : **≥ 80 %** — puis, pour les thèmes à épreuve, l'**évaluation pratique** de la plateforme ([evaluations/eval-<thème>.md](#) , seuil 14/20).

Les modules

#	Cours	Contenu	TD (corrigé)	TP
00	Fondamentaux	Binaire, électronique numérique, architecture CPU, mémoire, GPIO, UART/I2C/SPI/CAN, interruptions, toolchain	TD 00	—
01	Langage C	Le langage roi de l'embarqué : pointeurs, mémoire, opérations bit à bit, volatile , accès registres, bare-metal	TD 01	—
02	C++	POO, RAII, templates, C++ moderne appliqué à l'embarqué	TD 02	—
03	Arduino	Matériel, IDE, GPIO, PWM, ADC, capteurs, projets progressifs	TD 03	TP 1 — Station météo
04	VHDL & FPGA	Logique programmable, entités, process, machines d'états, testbenchs, Vivado/Quartus/GHDL	TD 04	TP 2 — UART VHDL
05	Java	POO complète, où Java sert dans l'industrie (supervision, Android, serveurs)	TD 05	—
06	Autres langages utiles	Assembleur ARM, MicroPython, Rust embarqué, Make/CMake, FreeRTOS, Linux embarqué	TD 06	—
07	Suite Siemens	TIA Portal, S7-1200/1500, LADDER, FBD, SCL, GRAFCET, WinCC, PROFINET	TD 07	TP 3 — Remplissage
08	Suite Schneider	EcoStruxure Control Expert & Machine Expert, M221/M241/M580, IEC 61131-3, HMI Harmony	TD 08	TP 4 — Cuve + Modbus
09	Parcours & ressources	Plan d'apprentissage sur 12 mois, projets, matériel à acheter, ressources gratuites	—	—
10	STM32	Le micro professionnel : STM32CubeIDE, HAL, clock tree, timers, DMA, NVIC, accès registres, debug SWD, FreeRTOS	TD 10	TP 5 — Météo FreeRTOS

Évaluations : [QCM par module \(56 questions, corrigé\)](#) · [Projets finaux notés avec barème /100](#)

Comment utiliser ce cours

1. **Commence par le module 00** même si tu es pressé : tout le reste s'appuie dessus (binaire, registres, protocoles).
2. **Le C avant tout le reste.** C'est la langue maternelle de l'embarqué : Arduino est du C++, les drivers Linux sont en C, le SCL de Siemens ressemble au Pascal/C. Maîtriser le C te donne 70 % du chemin.
3. **Pratique chaque module.** Chaque chapitre contient des exercices. Lire ne suffit jamais : tape le code, casse-le, répare-le.
4. **Matériel minimal recommandé** (~50–80 €) :
5. Un Arduino Uno ou Nano (clone à ~5 €)

6. Une breadboard + kit de composants (LED, résistances, boutons, capteurs)
7. Plus tard : une carte FPGA d'entrée de gamme (Tang Nano 9K ~15 €, ou Basys 3 si budget) et/ou un STM32 « Blue Pill » (~3 €)
8. **Pour les automates** (Siemens/Schneider), pas besoin d'acheter un automate : les deux éditeurs proposent des simulateurs (PLCSIM pour Siemens, le simulateur intégré de Control Expert / Machine Expert pour Schneider).

Vue d'ensemble : qui fait quoi ?

NIVEAU	LANGAGE / OUTIL
Supervision / SCADA	Java, Python, C#, WinCC, EcoStruxure
Automates (PLC)	LADDER, FBD, SCL, ST (IEC 61131-3)
Applications embarquées	C++, Rust, MicroPython
Firmware / drivers	C, un peu d'assembleur
Logique matérielle	VHDL, Verilog (FPGA / ASIC)
Silicium	Électronique numérique (portes, FF)

Plus on descend, plus on est proche du matériel, et plus le code décrit *ce que fait physiquement le circuit* plutôt que des instructions exécutées l'une après l'autre. Le VHDL, tout en bas, **ne s'exécute pas** : il *décrit* un circuit. C'est le changement de paradigme le plus important de ce cours.